📚 Rolling MCP Documentation

📚 Rolling MCP Documentation

Document snapshot: 8 November 2026

Source: release v1.0.1, commit `7bc2ea594b0b9d7f379f90b2f2f755a8ef47ee03`.

Repository copy: doc/rolling-mcp-doc.pdf, first added in PR #67.

Related work: SCRUM-197 Rolling Documentation · SCRUM-196 Delivery Execution · Delivery Plan · Handover & Acceptance Register.

🧭 Document guide

Section	Contents
1 · Baseline & prerequisites	Recorded source revision and dependency assumptions
2 · Installation	Composer install, module enablement and endpoint permissions
3 · OAuth2 configuration	Role, keys, scopes, client registration and redirect
4 · Rate-limit settings	Global/per-tool limits and initial default settings
5 · Connection & architecture	HTTPS, stdio, plugin factory, optional modules, tools and security controls
6 · Expected behaviour	Connection screenshots and tool discovery
7 · Troubleshooting	Connection, plugin, authentication and manual tool diagnostics
8 · DDEV verification & validation	15 DDEV procedures with preconditions, steps and expected results
9 · Historical issues	Known configuration and integration limitations

1. 📌 Source revision & prerequisites

Documented baseline: v1.0.1 (`7bc2ea5`). The module uses Composer with an external PDF parser dependency, Drupal configuration permissions, an MCP endpoint and optional tooling over HTTPS or stdio.

1.1 Sprint 6 regression fixes and current CI baseline

PR #61 restores unit/kernel tests, PR #63 fixes six defects, and PR #65 runs PHPUnit in CI. They were merged into `main` on 1 November 2026 and released as v1.0.1. Main CI #268 passed 124 tests / 1,830 assertions.

For setup and troubleshooting, `get_node` / `read_node_pdf` now enforce Drupal node `view` access; the OAuth `mcp_connector` role needs **View published content**, while Drush stdio uses anonymous access. `search_nodes` limits are 1–50; HTTP/stdio MCP tool failures set `isError`; long-running stdio rate limits use current time. Continue to verify live OAuth/stdio behaviour and the client environment. Sprint 6 verification record.

2. ⚙️ Installation & endpoint enablement

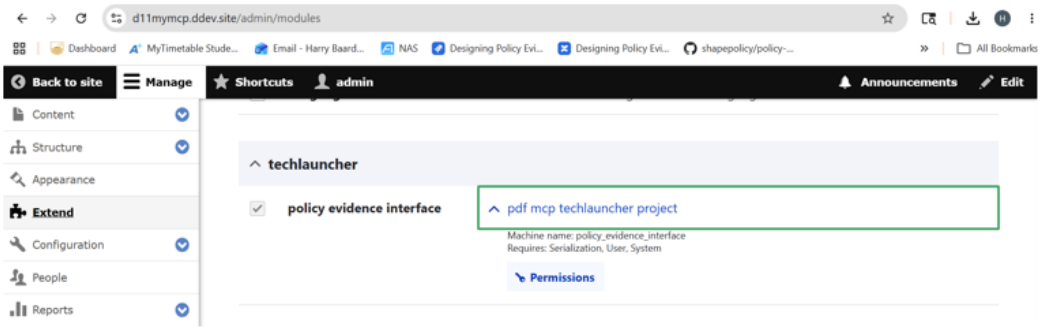
Attempt a composer install this should install the one external dependency that the project requires if not you will have to install smalot/parser manually with composer install.

In terminal navigate to Drupal project root then within run. (if using DDEV you need to append that to the head, further details at: 📦 Composer Setup)

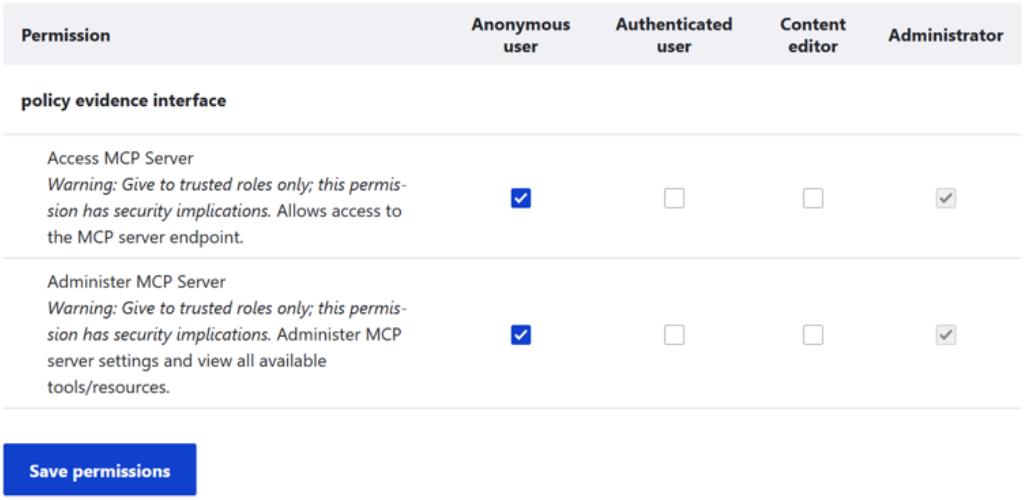
```
composer config repositories.policy-evidence-interface vcs
https://github.com/shapepolicy/policy-evidence-interface
```

```
composer require shapepolicy/policy-evidence-interface:^1.0
```

Install the module and dependencies clear the cache. Then you will need to edit the endpoint access permissions.



Click on the permissions button



enable access to the endpoint

After this clear the cache and you should be able to access the MCP endpoint.

3. 🔐 OAuth2 configuration

Confirm the configuration paths and permission names against the installed OAuth/Consumer module version.

3.1 Create the connector role and assign users

1. In **People → Roles**, create `mcp_connector`.
2. In **People**, assign that role only to users permitted to use the MCP endpoint.
3. In **Permissions**, ensure the required **Grant OAuth2 codes** permission is enabled for the relevant access configuration.

3.2 Consumer settings and keys

1. Open **Configuration → Web services → Consumers**, then the **Settings** tab.
2. Add the public/private key pair used by the Consumer setup. **Store private keys outside the web root and restrict filesystem permissions.**

3.3 Configure the OAuth scope

1. Open the **Scopes** tab and check the relevant grant permission.
2. Create a scope such as `mcp_connector_scope`.
3. Enable the **Authorization Code** and **Refresh Token** grant types; set granularity to **role** and associate the `mcp_connector` role.

3.4 Register the consumer/client

1. Open the **Clients** tab and create a consumer.
2. The original configuration calls for the consumer label `mcp connector`. Set a suitable client ID and enable authorization-code and refresh-token grants.
3. Assign the scope created above, enable **PKCE** and add the actual client redirect URI. For local development, a callback example is `http://127.0.0.1:10999/oauth/callback`; confirm the port and host on the actual local client. The documented Claude example is https://claude.ai/api/mcp/auth_callback.
4. The original example uses a **non-confidential** client (the confidential option unchecked). Enable the consumer and configure automatic authorisation, remembered approval and token expiry according to the site's policy.

OAuth callbacks, scopes and grants depend on the client; validate the configured environment.

4. ⏱️ Rate-limit configuration

Go to **Configuration → System → Policy evidence interface settings**.

To enable the system you will need to make sure that the rate limiter is enabled then input a global and tool rate limit greater than 0.

5. 🧩 Connector setup, features & architecture

5.0 Claude Desktop connection preparation

Remote HTTPS: in the Claude Desktop client, add the server HTTPS endpoint as a custom connector. **Local HTTPS:** install Node.js and use the developer settings configuration with the documented `npx mcp-remote` proxy. **Local stdio:** run Drush from the Drupal site root (the source's WSL/DDEV sample includes `--cd` and the local project path). Verify actual client configuration and OAuth callback/permissions.

5.1 HTTPS & stdio connector examples

MCP endpoint connection through https is possible requires npx to launch proxy server:

```
"mcpServers": {
    "drupalLocalHttp": {
      "command": "npx",
      "args": [
        "mcp-remote",
        "https://d11mymcp.ddev.site/_mcp"
      ],
      "env": {
        "NODE_OPTIONS": "--use-system-ca"
      }
    }
}
```

TLS safety: `NODE_TLS_REJECT_UNAUTHORIZED=0` disables certificate verification and must not be used in production or with sensitive credentials. Trust the development CA instead.

```
"mcpServers": {
    "drupalLocalHttp": {
      "command": "cmd",
      "args": [
        "/c",
        "npx",
        "-y",
        "mcp-remote",
        "https://20261005d11.ddev.site/_mcp"
      ],
      "env": {
        "NODE_TLS_REJECT_UNAUTHORIZED": "0"
      }
    }
  }
```

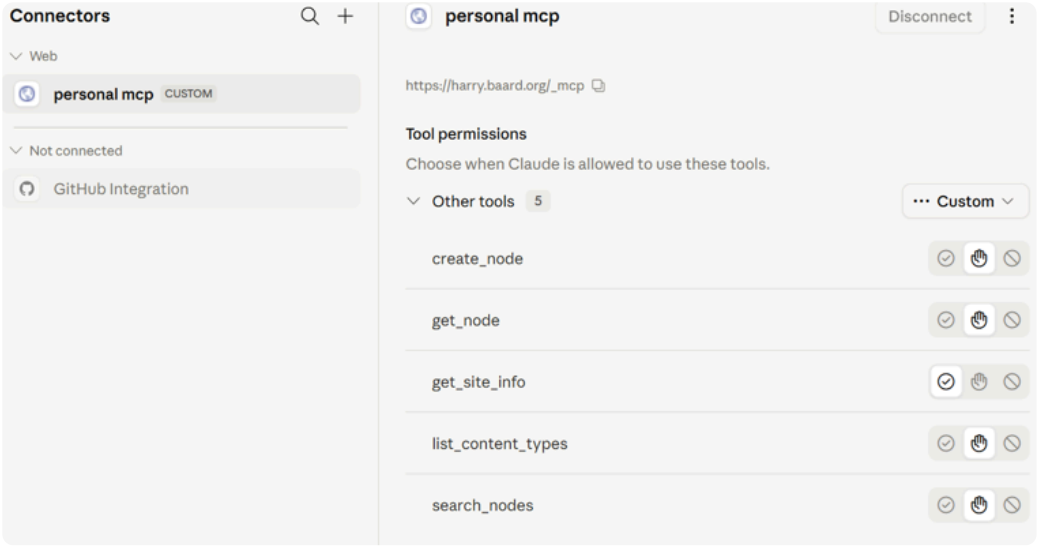
The alternative local HTTP-proxy configuration can be used when the primary connector fails.

```
"mcpServers": {
    "drupalLocalStdio": {
      "command": "wsl",
      "args":[
        "--cd", "/home/harry/workspace/d11mymcp",
        "ddev", "exec", "vendor/bin/drush", "mcp:server"
```

```
        ]
      }
    }
```

The above does the studio

5.2 Global connector example



5.3 MCP tool/resource plugin factory

Allows for the easy addition of new tools and resources into the module. Using Drupal core plugins. **WARNING** initial **comment** is **effects code behavior** as it's how the Drupal core/plugin discovers the tool/resource and what data is pulled in such as description. See example comment block below. Make sure that the plugins are put into the correct folder.

```
/**
 * Returns basic information about the Drupal site.
 *
 * @McpTool(
 *   id = "get_site_info",
 *   tool_name = "get_site_info",
 *   description = "Returns general information about this Drupal site including name, slogan, Drupal version, and base URL."
 * )
 */
```

Below both resource and tool have the same output but the resource appears to be partially cached into the LLM so that it doesn't need to query server over and over.

Tool Example: policy-evidence-interface/src/Plugin/McpTool/GetSiteInfo.php at main · shapepolicy/policy-evidence-interface

Resource Example: policy-evidence-interface/src/Plugin/McpResource/SiteInfo.php at main · shapepolicy/policy-evidence-interface

5.4 Optional tool/resource submodules

Allows for the easy addition of sub modules containing tools or resources. So that many tools can be bundled together as a module or so that more complex tools can

have activation and deactivation hooks associated e.g rank vector search.

Sub module Example: policy-evidence-interface/modules/policy_evidence_interface_basic_tools at main · shapepolicy/policy-evidence-interface * only contains one tool

5.5 OAuth-backed MCP access control

Uses a mocked dynamic client register point to allow users to get OAuth tokens from the configured OAuth consumer. Over https the user only needs to provide the website URL, the user will then be prompted to login if they aren't into the website and then grant the associated OAuth grant to enable MCP usage.

The OAuth is used to identify the user that is using the connector via the token that they pass. This allows the user ID to be found. This is currently used in the MCP system to get the user roles and based access control. To allow or deny MCP access based on the user having the mcp_connector rule.

This can be further extended to allow the system to have tiered access based on the users role. e.g a user with role a can use a wider set of tools or use the tools more than a user with role b.

5.6 Global taxonomy-tag exclusion

Uses Drupal core tags system, to allow the blocking of bodies of work containing target tags such as **NO AI**, **copyright**. On module install no tags will be blocked. The tags can be added/removed via the **Policy evidence interface settings** form through tag IDs.

The tag blocking logic has been applied to the preset tools search_nodes get_node and read_node_pdf to hide or disable the accessing of associated documents via the tools.

5.7 Global and per-tool rate limits

The rate limiter settings can be accessed via the **Policy evidence interface settings** form. The rate limiter is used to limit the number of calls an user make both globally and tool wise.

The rate limiter can be used to disable mcp system or tool calls by disabled by un-checking the Enable rate limiting or by setting the limit to 0.

The rate limiter window size can be adjusted by seconds to allow n number of calls every 60 or 30 seconds.

The rate limiter form will be dynamically updated based on the present tools. So if a tool is added/removed the rate limit will adjust. Tools that are added will initialize at 0 so that they won't be immediately usable/callable.

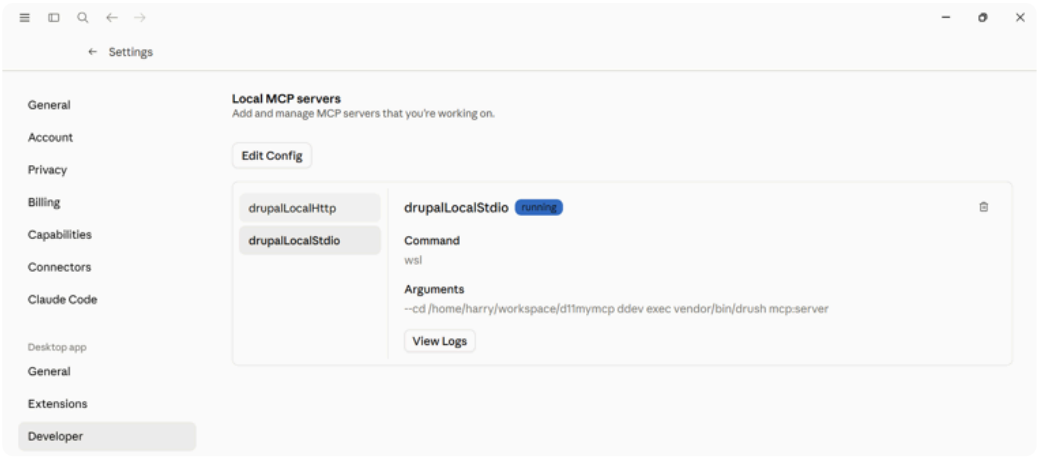
5.8 Preset MCP tools

- `get_site_info` — returns general site info (name, version, URL, etc.)

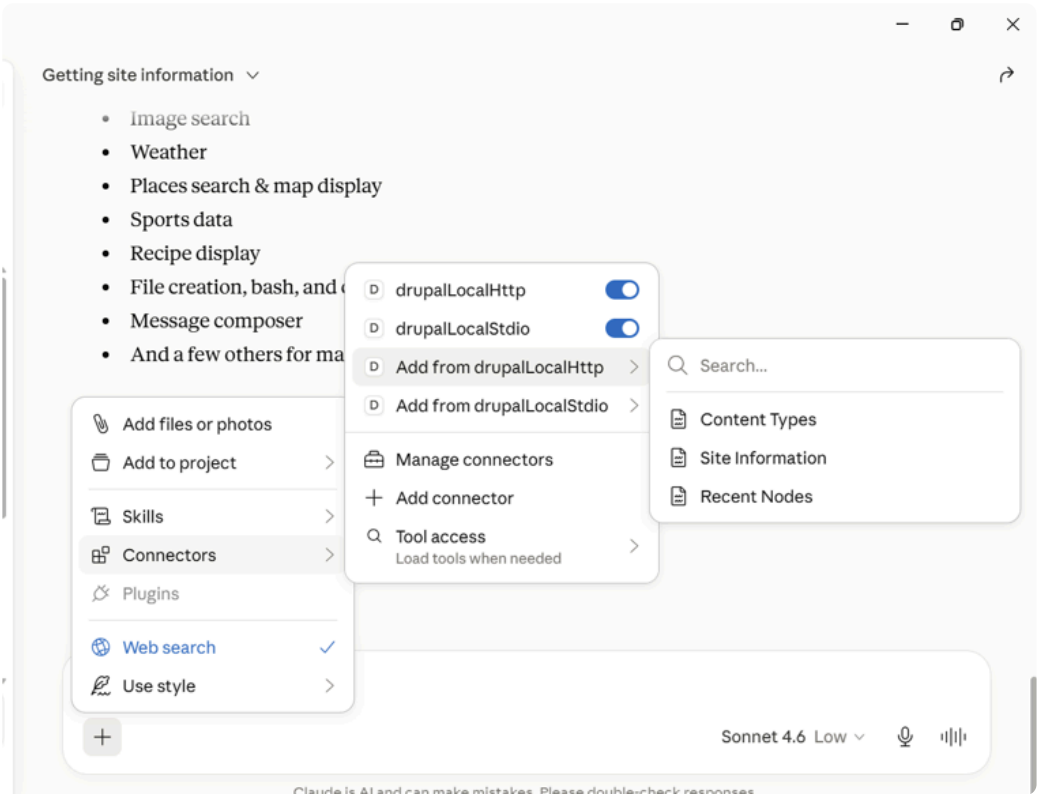
- `list_content_types` — lists all content types on the site
- `search_nodes` — searches published nodes by keyword, optionally filtered by content type
- `get_node` — fetches full details of a node by its ID
- `read_node_pdf` — extracts text from a PDF attached to a node

6. ✅ Expected behaviour & screenshots

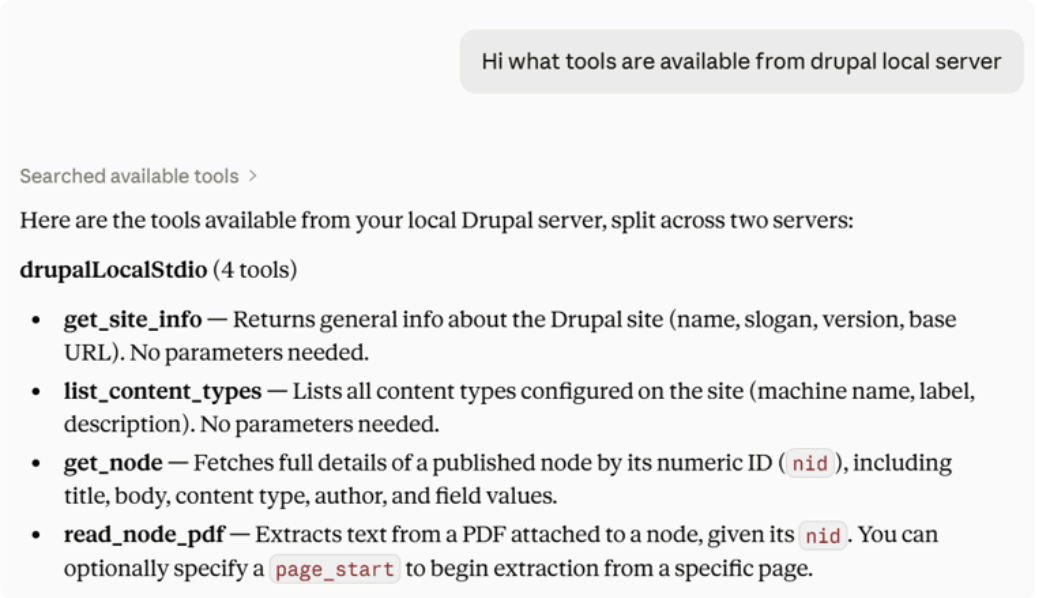
connection is connected and working:



able to see server is running when viewing developer



Is able to see resources from the server



Is able to access available tools and return json-rcp description interpretation

7. 🛠️ Troubleshooting & diagnostics

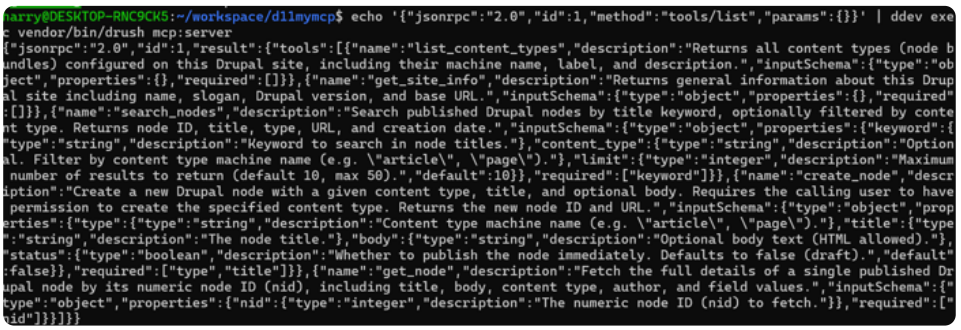
7.1 Server disconnects or fails to start

Server is not connecting i.e Claude is returning that it is disconnecting. The server is either actually disconnecting or isn't connecting at all you will need to open up the Claude developer options and examine the logs for what is breaking 🙂.

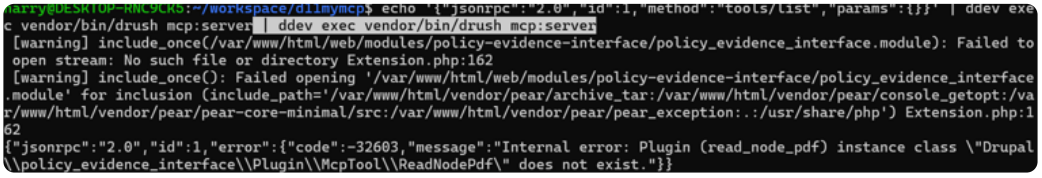
7.2 Server connected but tools are missing

This is an issue with one of the plugins to debug what plugin and how you will need to run below this will output the available tools that the MCP has access to.

```
echo '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}' |
ddev exec vendor/bin/drush mcp:server
```



nominal output



This will output any PHP Errors useful for debugging. In this case the issue was caused by not rebuilding cache after changes.

7.3 Manual tool/resource testing

If the tools are loading correctly you may need to manually test it out see below code snippet running the read node PDF tool:

policy-evidence-interface/src/Plugin/McpTool/ReadNodePdf.php at main · shapepolicy/policy-evidence-interface

```
harry@DESKTOP-RNC9CK5:~/workspace/d11mymcp$ curl -X POST
https://d11mymcp.ddev.site/_mcp -H "Content-Type: application/json" -d
"{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/call\",\"params\":
{\"name\":\"read_node_pdf\",\"arguments\":
{\"nid\":4,\"page_start\":1}}}"
```

7.4 Tool name/branch mismatch

Make sure the expected version is installed (`composer show shapepolicy/policy-evidence-interface`) and you have rebuilt the cache.

7.5 Authentication failures

Invalid scope:

make sure that you spell the scope correctly

Invalid authorization:

make sure that the URI redirect link is correct.

Able to all drupal server authorisation grant but failing on claude refucing to connect or connection failure:

You should see the tokens being generated. But claude is refusing to take em.

You may need to go to cloud flare or equilivent and disable DNS Proxy.

Passed all but unable to connect or use tools:

make sure that you have assinged a `mcp_connector` role to the associated user.

8. 🧪 DDEV Verification & Validation (23 October 2026)

Verification status: The 15 DDEV procedures include preconditions, steps and expected results; individual observed outcomes and logs are not recorded. In TC-03, confirm the intended `list_content_types` call because the example prompt requests site information.

Case	Scenario
TC-01	Connect and discover available tools
TC-02	Get site information

TC-03	List content types
TC-04	Retrieve a node
TC-05	Read a node PDF
TC-06	Search nodes
TC-07	Revoke OAuth connector role
TC-08	Exclude one tagged item from search
TC-09	Exclude several taxonomy tags
TC-10	Block retrieval of a tagged node
TC-11	Block PDF retrieval of a tagged node
TC-12	Global rate-limit window
TC-13	Per-tool rate-limit threshold
TC-14	Disable a tool using limit zero
TC-15	Discover the basic-tools submodule

TC-01 · Connect and discover available tools

Purpose: Open a custom connector

Preconditions: You have claude installed and installed the module into a drupal server and have followed installation steps. You can use Either studio or http connector.

Steps:

1. Open claude.
2. Prompt “show me available tools on the custom connector“

Expected result: you may be prompted to log into the website and grant oAuth token to calude. You should see the available tools listed by Claude. If you click on the plus button you should also see avaible

TC-02 · Get site information

Purpose: Tool get_site_info

Preconditions: You have been able to perform TC 1. You have enabled the rate limits to at least allow for one call.

Steps:

1. Open claude.
2. Prompt “Hi can you get the site information of the custom connector“

Expected result: you may be prompted to log into the website and grant oAuth token to calude. You should see the site informaiton.

TC-03 · List content types

Purpose: Tool List contentent types

Preconditions: You have been able to perform TC 1. You have enabled the tool via rate limits and have a articles inside of the website.

Steps:

1. Open claude.
2. Prompt “Hi can you get the site information of the custom connector“

Expected result: you may be prompted to log into the website and grant oAuth token to calude. You should see the site informaiton.

TC-04 · Retrieve a node

Purpose: Tool get node

Preconditions: You have been able to perform TC 1. You have enabled the tool via rate limits and have a articles inside of the website. You have a valid node at x. The node uses a valid article strucuture.

Steps:

1. Open claude.
2. Prompt “Hi can you get the information of node x“

Expected result: you may be prompted to log into the website and grant oAuth token to calude. You should see the information of the node returned.

TC-05 · Read a node PDF

Purpose: Tool read pdf node

Preconditions: You have been able to perform TC 1. You have enabled the tool via rate limits and have a articles inside of the website. You have a valid node at x. The node uses a valid article strucuture. The node has an pdf that is readable by smallot parser

Steps:

1. Open claude.
2. Prompt “Hi can you read the pdf of node x“

Expected result: you may be prompted to log into the website and grant oAuth token to calude. You should see the information of the pdf returned.

TC-06 · Search nodes

Purpose: search nodes tool

Preconditions: You have been able to perform TC 1. You have enabled the tool via rate limits and have a articles inside of the website. You have a valid node contents at x. The node uses a valid article strucuture. There are available nodes to search for.

Steps:

1. Open claude.
2. Prompt “Hi can you find search for x“

Expected result: you may be prompted to log into the website and grant oAuth token to calude. You should see a list of relevant nodes found.

TC-07 · Revoke OAuth connector role

Purpose: Oauth access control

Preconditions: You have been able to perform TC 1. you have granted an oauth token.

Steps:

1. Go to people and remove mcp_connector role from the user.
2. Open claude
3. Prompt “Hi can connect to the custom connector“

Expected result: claude will say that it is unable to connec to the connector or when propted to connect to the server, caldue will hang and will freeze for a while before saying that it was a unable to connect.

TC-08 · Exclude one tagged item from search

Purpose: tag based access control for search tool

Preconditions: You have been able to perform TC 1. you have granted an oauth token. you have a valid tag that is assocated with an valid article. You have the tag ID. you have a promt that would find the articale if it wasn’t disallowed.

Steps:

1. Go to **Configuration → System → Policy evidence interface settings** → add the tag to the disallowed tag IDs.
2. Open claude
3. Prompt “Hi can can you search for x“

Expected result: claude will be unable to find the target articles.

TC-09 · Exclude several taxonomy tags

Purpose: tag based access control for many disallowed tags

Preconditions: You have been able to perform TC 1. you have granted an oauth token. you have a valid tags that is assocated with an valid articles. You have the tag ID. you have a promt that would find the articale if it wasn't disallowed.

Steps:

1. Go to **Configuration → System → Policy evidence interface settings** → add the tags to the disallowed tag IDs.
2. Open claude
3. Prompt "Hi can can you search for x"

Expected result: claude will be unable to find the target articles.

TC-10 · Block retrieval of a tagged node

Purpose: tag based access control for read node

Preconditions: You have been able to perform TC 1. you have granted an oauth token. you have a valid tags that is associated with an valid article. You have the tag ID. You have an article that node ID that should be disallowed.

Steps:

1. Go to **Configuration → System → Policy evidence interface settings** → add the tags to the disallowed tag IDs.
2. Open claude
3. Prompt "Hi can can you read node x"

Expected result: claude will be unable to access the article.

TC-11 · Block PDF retrieval of a tagged node

Purpose: tag based access control for read node pdf

Preconditions: You have been able to perform TC 1. you have granted an oauth token. you have a valid tags that is associated with an valid article which contains a pdf. You have the tag ID. You have an article that node ID that should be disallowed.

Steps:

1. Go to **Configuration → System → Policy evidence interface settings** → add the tags to the disallowed tag IDs.
2. Open claude
3. Prompt "Hi can can you read node x pdf"

Expected result: Claude will be unable to access the article's pdf.

TC-12 · Global rate-limit window

Purpose: rate limiter global window and max tool calls check

Preconditions: You have been able to perform TC 1. Global tool limit is set to 1

Steps:

1. Go to **Configuration → System → Policy evidence interface settings** → change the window duration to x.
2. Open claude
3. Prompt “Hi can can you get site info twice“

Expected result: Claude will be able to call the tool once. further calls will prompt Claude that it is unable to call again until x seconds in window has passed. After x seconds has passed you will be able to call get site info again.

TC-13 · Per-tool rate-limit threshold

Purpose: rate limiter tool wise limiter

Preconditions: You have been able to perform TC 1. Global tool limit is set to 10. There is another valid tool that can be called.

Steps:

1. Go to **Configuration → System → Policy evidence interface settings** → change the get_site_info rate limit to 1.
2. Open claude
3. Prompt “Hi can can you get site info“

Expected result: Claude will be able to call the tool once then will say that it is unable to call the tool again. If you call a different tool you will be able to call it without waiting for the window to reset.

TC-14 · Disable a tool using limit zero

Purpose: rate limiter tool wise disable

Preconditions: You have been able to perform TC 1. Global tool limit is set to 10. Get site info has been set to 0 there is another tool that can be called.

Steps:

1. Open claude
2. Prompt “Hi can can you get site info“

Expected result: Claude will be unable to call the tool and will reply back that the tool seems disabled. If you call a different tool you will be able to do so.

TC-15 · Discover the basic-tools submodule

Purpose: enable basic tool sub modules

Preconditions: You have been able to perform TC 1. Ensure that the submodule is installed.

Steps:

1. Open claude
2. Prompt “Hi can can you list avaiable tools“

Expected result: Claude will list the tool stored in basic tools sub module. If you open **Configuration → System → Policy evidence interface settings** and scroll down you should see the tools added by the module is now configurable in the rate limiter.

Delivery evidence: Record tester, DDEV/Drupal/PHP versions, commit, actual response, status, logs/screenshots and any client-specific deviation when repeating these cases for handover. Related records: Final Verification & Demonstration Report and Handover & Acceptance Register.

9. ⚠️ Known issues & historical limitations

Review the following configuration and integration limitations against the final deployed revision. The earlier CI failure is superseded by Sprint 6 PHPUnit CI evidence.

Area	Issue / limitation
Historic CI status	Earlier CI was unreliable; Sprint 6 PR #65 introduced PHPUnit CI, and branch run #246 passed. Confirm the final integrated revision.
Hard-coded field names	Tool inflexibility tools are hard-coded to article names consumer names. I.e if an article body has a machine code value different to body_field. Note that this will only cause a soft failure client and server side.
Rate-limit persistence	If a tool is added and the rate limiter is set then the tool is removed. When the tool is readded the rate limit won’t be set to 0. To fix it set it so that if an field array isn’t pickup during the form submit action it will remove the tool entry from the settings.
OAuth redirect and connector constraints	OAuth redirect URI GPT and other LLMs shareing the same API as claude arn’t specific enough about the redirect URI that they use. They also pay wall connecting

	custom connectors onto their app so testing can't be performed.
DNS proxy/authentication	DNS Proxy issues. When developing the OAuth application and migrating to hosted enviroment to test after testing on local environment it was found that DNS Proxy may interfeir with the autharisation processes primerally on the LLM side to accept the token generated.
Tool-level permission enforcement	Remaining security work is validation and hardening of tool-level permission enforcement, including testing against JSON-RPC input manipulation and unintended privilege escalation.
PDF parser dependency	Dependancy you will need to composer require the file for the pdfparser to dependancies to be loaded in.
Plugin failure resilience	fragile if a tool/resource doesn't work it will break the MCP connection (behavior of Drupal plugins) see debug for help with getting useful debug.

Related risk status: Delivery Risks & Assumptions and the documented test evidence in Final Verification & Demonstration Report. Outstanding items require verification against the handover revision.